

```

53         </div>
54         <div th:each="c : ${product.comments}" class="comment">
55             <span th:text="${c.text}">Comment Content</span>
56         </div>
57     </div>
58 </div>
59
60 </body>
61 </html>
62

```

productdetail.html

```

1  <!DOCTYPE html>
2  <html lang="en" xmlns:th="http://www.thymeleaf.org">
3  <head>
4      <meta charset="UTF-8">
5      <title>Title</title>
6      <style>
7          .comments {
8              margin-top: 10px;
9              padding-left: 20px;
10         }
11
12         .comment {
13             border-left: 3px solid #2a7;
14             margin-bottom: 8px;
15             padding-left: 10px;
16             font-style: italic;
17         }
18     </style>
19 </head>
20 <body>
21     <h1>Product Detail</h1>
22     <p th:text="'Name: ' + ${product.name}"></p>
23     <p th:text="'Price: ' + ${product.price}"></p>
24     <div class="comments">
25         <h3>Comments:</h3>
26         <div th:if="${#lists.isEmpty(product.comments)}">
27             <p><em>Nothing</em></p>
28         </div>
29         <div th:each="c : ${product.comments}" class="comment">
30             <span th:text="${c.text}">Comment Content</span>
31         </div>
32     </div>
33
34 </body>
35 </html>

```

Tuần 8-9: Spring Security

Thiết lập 3 role cho hệ thống:

1. Guest: người dùng không có tài khoản, có chức năng Đăng ký tài khoản, xem danh sách Product.
2. Customer: người dùng có tài khoản, có chức năng Xem danh sách Product, mua Product, Lưu Order, tra cứu Order, OrderLine
3. Admin: Người quản trị hệ thống có đầy đủ các chức năng quản trị Customer, Product

Các bước thực hiện thiết lập spring Security:

Bước 1: Add Dependency

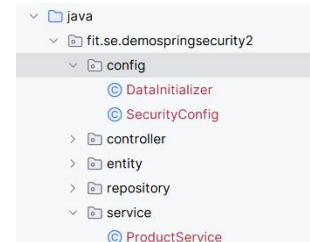
```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Bước 2: Tạo **config** package in project

→ Tạo **SercurityConfig** class và add annotation

Code Gợi ý cho 2 role: ADMIN, CUSTOMER,..

Trong **SercurityConfig**, tạo a bean trả về **SecurityFilterChain** có tham số là **HttpSecurity** cho phép thực hiện các filter methods tương ứng với các Role



SecurityConfig.java

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SercurityConfig {
    @Bean
    public BCryptPasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public UserDetailsService userDetailsService() {
        UserDetails admin = User.builder()
            .username("admin")
            .password(passwordEncoder().encode("123"))
            .roles("ADMIN")
            .build();

        UserDetails customer = User.builder()
            .username("customer")
            .password(passwordEncoder().encode("111"))
            .roles("CUSTOMER")
            .build();

        return new InMemoryUserDetailsManager(admin, customer);
    }

    @Bean
    public SecurityFilterChain securityFilterChain (HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                //1. Quy tắc mở
```

```

        .requestMatchers("/login", "/css/**", "/js/**", "/images/**",
"/test").permitAll()
        //2. Phân quyền

.requestMatchers("/product", "/product/detail/**").hasAnyRole("CUSTOMER", "ADMIN")
        //3. Đọc quyền
        .requestMatchers("/product/add", "/product/edit/",
"/product/update/").hasRole("ADMIN")
        //4. phần request còn lại phải chứng thực
        .anyRequest().authenticated()
    )
    .formLogin(form -> form
//        .loginPage("/login")
        .defaultSuccessUrl("/product")
        .permitAll()
    )
    .logout(logout -> logout
        .logoutUrl("/logout") // URL người dùng truy cập để đăng xuất
        (mặc định là POST)
        .logoutSuccessUrl("/login?logout=true") // Chuyển hướng sau khi
        đăng xuất thành công
        .invalidateHttpSession(true) // Hủy session
        .deleteCookies("JSESSIONID") // Xóa cookie session
        .permitAll() // Cho phép tất cả mọi người truy cập vào URL logout
    );

    return http.build();
}
}

```

Bước 3: Thiết lập các Mapping cho các Methods trong Controller với các role đã config:

Annotation cho role tương ứng:

Code gợi ý:

ProductController.java

```

@Controller
@RequestMapping("/product")

public class ProductController {

    @Autowired
    private ProductService productService;

    @Autowired
    private CategoryService categoryService;

    @GetMapping()
    public String showProducts(Model model) {

        model.addAttribute("products", productService.findAll());
        return "product/list";
    }

    @PreAuthorize("hasRole('ADMIN')")
    @GetMapping("/add")
    public String showAddForm(Model model) {
        model.addAttribute("product", new Product());
    }
}

```

```

        model.addAttribute("categories", categoryService.findAll());
        return "product/productform";
    }
    @PreAuthorize("hasRole('ADMIN')")
    @PostMapping("/save")
    public String saveProduct(@ModelAttribute Product product,
                              @RequestParam("fileImage") MultipartFile file
                              ) throws IOException {

        if (!file.isEmpty()) {
            //      String fileName = file.getOriginalFilename();
            String fileName = UUID.randomUUID() + "_" + file.getOriginalFilename();
            //      Path uploadDir = Paths.get("uploads/");
            Path uploadDir = Paths.get("src/main/resources/static/uploads/");

            Files.createDirectories(uploadDir);
            file.transferTo(uploadDir.resolve(fileName));
            product.setImage(fileName);
        }
        productService.save(product);
        return "redirect:/product";
    }

    // Show edit form
    @PreAuthorize("hasRole('ADMIN')")
    @GetMapping("/edit/{id}")
    public String showEditForm(@PathVariable("id") Long id, Model model) {
        Product product = productService.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("Invalid product Id: " +
id));

        model.addAttribute("product", product);
        model.addAttribute("categories", categoryService.findAll());
        return "product/editproduct";
    }

    // Handle update
    @PreAuthorize("hasRole('ADMIN')")
    @PostMapping("/update")
    public String updateProduct(@ModelAttribute("product") Product product,
                                @RequestParam("fileImage") MultipartFile file) throws
IOException {

        // Handle new image upload (optional)
        if (!file.isEmpty()) {
            //      String fileName = file.getOriginalFilename();
            String fileName = UUID.randomUUID() + "_" + file.getOriginalFilename();
            //      Path uploadDir = Paths.get("uploads/");
            Path uploadDir = Paths.get("src/main/resources/static/uploads/");

            Files.createDirectories(uploadDir);
            file.transferTo(uploadDir.resolve(fileName));
            product.setImage(fileName);
        } else {
            // keep existing image
            Product existing = productService.findById(product.getId()).orElse(null);
            if (existing != null) {
                product.setImage(existing.getImage());
            }
        }

        productService.save(product);
    }

```

```

        return "redirect:/product";
    }

@GetMapping("/detail/{id}")
public String showProductDetails(@PathVariable("id") Long id, Model model) {
    Product p=productService.findById(id).orElse(null);
    model.addAttribute("product", p);
    return "product/productdetail";
}
}

```

Bước 4: Tại View thiết lập các role cần gọi các config cho các action:

Code gợi ý:

```

<div sec:authorize="hasRole('ADMIN')">
    <a th:href="@{/products/add}" class="btn btn-primary">
        <i class="fas fa-plus"></i> Add New Product
    </a>
</div>

```